COMPUTING 2 - PARALLEL COMPUTING

Chapter 2 Mock Test

CMP, Superscalar, SMT, SIMD, and Memory

Suggested time	Total points	Coverage
40 minutes	50	ILP, thread-level parallelism, vector execution, coherence, consistency, false sharing

NAME		STUDENT ID	
DATE		SCORE	/ 50

Instructions
- Answer every question and show essential reasoning for partial credit.
- For true/false items, an unsupported letter receives no credit.
- Unless stated otherwise, assume NVIDIA warp size 32 and ideal arithmetic.
- Keep the answer section closed until the suggested time has elapsed.

Questions

2.1 Parallelism taxonomy — 8 points

Compare superscalar execution, SIMD/vector execution, SMT, and multicore. For each, name the parallel unit and one resource relationship.

WORK AREA

2.2 Vector instruction count — 7 points

A loop performs one element-wise add for 1024 elements. A vector unit processes 8 elements per vector instruction. Ignore cleanup and loads/stores.

(a) How many scalar add instructions are replaced? (b) How many vector add instructions are issued? (c) Why can this reduce front-end pressure?

WORK AREA

2.3 Dynamic issue and out-of-order execution — 7 points

Explain what a superscalar out-of-order core decides dynamically. Why can it still stall even when the issue width is four?

WORK AREA

2.4 False sharing diagnosis — 10 points

Eight threads repeatedly update counters[tid], where each counter is an 8-byte integer and the cache line is 64 bytes. Explain the performance problem even though no two threads update the same element. Give two fixes.

```
long counters[8];
#pragma omp parallel
for (int k=0; k<10000000; ++k)
    counters[omp_get_thread_num()]++;
```

WORK AREA

2.5 Coherence versus consistency — 8 points

Define cache coherence and memory consistency. Why does coherence alone not specify all legal outcomes of a multithreaded program?

WORK AREA

2.6 SMT tradeoff — 10 points

Two threads run on one SMT core. Thread A is compute-bound and saturates arithmetic units; thread B has frequent long cache misses. Predict when SMT helps and when it may hurt. Contrast this with placing the threads on separate cores.

WORK AREA

ANSWER SECTION

Chapter 2: Answer Key & Worked Solutions

Grade the method as well as the final result. Numerical answers show enough intermediate work to identify where an error occurred.

Quick Answer Key

Q	Topic	Short answer
2.1	Parallelism taxonomy	Superscalar: instructions; SIMD: lanes/elements; SMT: threads sharing one core; multicore: separate cores
2.2	Vector instruction count	1024 scalar adds -> 128 vector adds; about 8x fewer arithmetic instructions to fetch/decode
2.3	Dynamic issue and out-of-order execution	Find ready independent instructions; stalls from dependencies, cache misses, branch recovery, or unavailable units
2.4	False sharing diagnosis	All counters share one line; coherence ping-pong. Pad/align per-thread data or accumulate privately then combine.
2.5	Coherence versus consistency	Coherence orders writes to one location; consistency constrains ordering across locations/operations
2.6	SMT tradeoff	SMT helps fill idle slots during B's misses but may contend with A; separate cores reduce pipeline contention

Worked Solutions

2.1 Parallelism taxonomy — 8 points

Answer. The four mechanisms expose different forms of parallelism and therefore are not interchangeable.

Mechanism	Parallel unit	Resource relationship
Superscalar	Independent scalar instructions	One core dynamically issues multiple instructions
SIMD/vector	Data elements/lanes	One instruction controls many lanes
SMT	Hardware threads	Threads share one core's pipelines and caches
Multicore/CMP	Cores	Each core has its own execution resources; some caches/interconnect are shared

2.2 Vector instruction count — 7 points

Answer. Vector length 8 means one arithmetic instruction describes eight independent additions.

- Scalar arithmetic instruction count=1024.
- Vector arithmetic instruction count=1024/8=128.
- The front end fetches and decodes fewer instructions, while the lanes perform the data-parallel work.

Common trap. This count ignores vector loads/stores and setup. It is not a claim that total runtime improves by exactly 8x.

2.3 Dynamic issue and out-of-order execution — 7 points

Answer. The core tracks dependencies and resource readiness, then issues multiple ready instructions from a window each cycle.

- A four-wide issue limit is only a maximum.
- True data dependencies reduce instruction-level parallelism.
- Cache misses, branch mispredictions, and functional-unit conflicts can leave fewer than four ready instructions.

2.4 False sharing diagnosis — 10 points

Answer. The eight 8-byte counters occupy exactly one 64-byte cache line. Writes by different cores repeatedly invalidate the same line.

- This is false sharing: different words, one coherence unit.
- Fix 1: place each counter in its own cache line using padding/alignment.
- Fix 2: keep the counter in a register/private variable during the loop and write once, followed by a reduction.

Common trap. Coherence operates at cache-line granularity, not variable granularity.

2.5 Coherence versus consistency — 8 points

Answer. Coherence is a per-address property; consistency is a global ordering contract visible to software.

- Coherence ensures writes to one cache line/location are eventually seen in an agreed order.
- Consistency defines which interleavings and reorderings across different addresses are legal.
- A system can be coherent yet allow surprising cross-address orderings unless synchronization establishes happens-before relationships.

2.6 SMT tradeoff — 10 points

Answer. SMT is most useful when the threads' stalls and resource demands are complementary.

- While B waits for memory, A can use issue slots that would otherwise be idle.
- If A already saturates execution ports, adding B may create contention for the front end, caches, registers, queues, or bandwidth.
- Separate cores provide more independent execution resources but still may share the last-level cache and memory bandwidth.